

JSON-to-Python Class Converter

Product Requirements Document (PRD)

Field	Detail
Project type	Agentic AI course project
Version	1.0 (Draft)
Date	28 September 2026
Output formats	Python Dataclass, Pydantic Model
Primary interface	Web UI (Streamlit) with optional CLI

1. Overview

The JSON-to-Python Class Converter is an agentic AI tool that takes a raw JSON object and produces ready-to-use Python class definitions, as either standard-library **dataclasses** or **Pydantic models**. Unlike a fixed script, an AI agent inspects the JSON, decides how it should be modelled, generates code, **runs that code against the original JSON to verify it**, and repairs its own output when something fails.

1.1 Problem statement

Developers who consume APIs or config files often receive JSON and must hand-write matching Python classes. This is repetitive and error-prone, especially with deeply nested objects, optional fields, and inconsistent arrays. Existing rule-based converters copy the structure literally but cannot make judgement calls such as choosing good class names, spotting dates or enums, or reconciling arrays whose items differ.

1.2 Goals

- Convert any valid JSON object into correct, idiomatic Python classes in one step.
- Support both Dataclass and Pydantic v2 output from the same analysis.
- Guarantee correctness by validating generated code against the input JSON before returning it.
- Make sensible modelling decisions (naming, nullability, unions, dates) and explain them to the user.
- Keep the experience simple: paste JSON, pick a format, get code.

1.3 Non-goals (v1)

- Generating classes from JSON Schema, OpenAPI, or XML/YAML inputs.
- Output in other languages (TypeScript, Java, Go).
- Generating serialization/deserialization business logic beyond what the chosen library provides.
- Hosting user data long-term or supporting multi-user accounts.

2. Why This Is an Agentic AI Project

A plain converter is a deterministic function: JSON in, code out. This project is agentic because the AI system works toward a goal (*a correct class that fits the user's JSON*) by choosing actions, using tools, observing results, and iterating.

Agentic property	How this project satisfies it
Goal-directed	The agent's goal is 'produce classes that successfully parse the user's JSON', not just 'emit text'.
Tool use	Calls tools: JSON parser, type inferencer, name resolver, code generator, code executor/validator.
Observe and act loop	Runs generated code, reads errors or mismatches, and revises the schema plan.
Autonomous decisions	Chooses class names, int vs float, Optional vs required, Union vs Any, date/UUID detection.
Self-correction	Retries up to N times using validator feedback before reporting a result.
Interaction when uncertain	Asks the user a clarifying question for genuinely ambiguous cases instead of guessing.
Explainability	Returns a short decision log describing the choices it made.

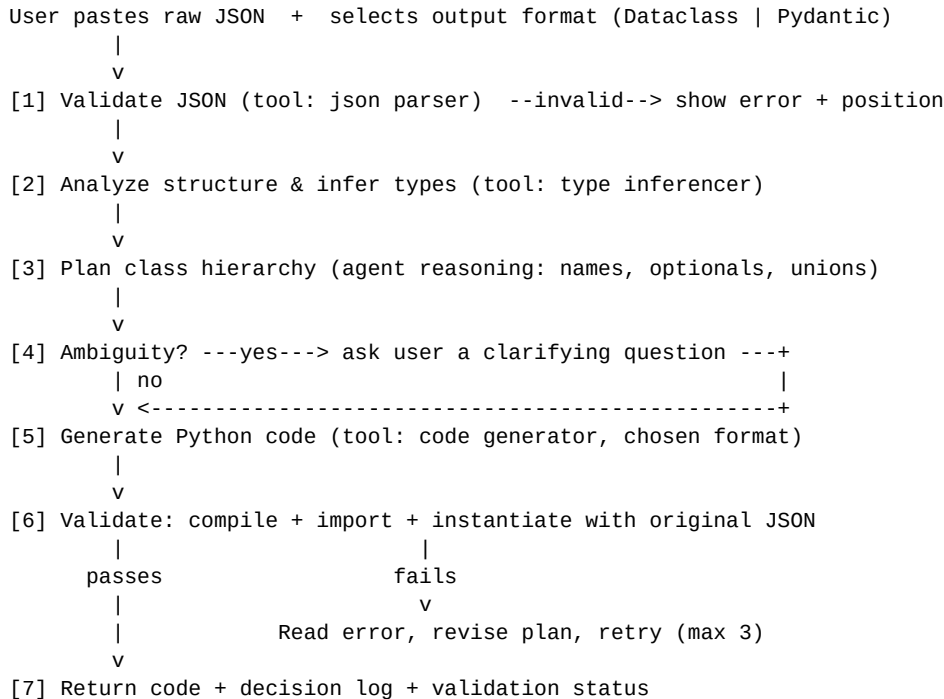
3. Target Users and Use Cases

User	Need
Backend / API developer	Turn third-party API responses into typed Python models quickly.
Data engineer	Model messy JSON payloads from logs, events, or webhooks.
Student / learner	Understand how JSON maps onto Python classes; compare Dataclass vs Pydantic.
Test / QA engineer	Generate typed fixtures from sample payloads.

3.1 Key user stories

- As a developer, I paste a JSON object and receive a working Python class so I do not write boilerplate.
- As a developer, I choose between Dataclass and Pydantic so the output matches my project.
- As a developer, I see what the agent decided and why, so I can trust or adjust the result.
- As a developer, I can supply several sample JSON objects so optional fields are detected correctly.
- As a developer, I get a clear message when my JSON is invalid rather than a confusing failure.

4. User Workflow



5. Functional Requirements

ID	Requirement	Priority
FR-1	Accept raw JSON text via text area (web) or stdin/file (CLI).	Must
FR-2	Validate JSON syntax and report line/column of any error.	Must
FR-3	Let the user select output format: Dataclass or Pydantic v2.	Must
FR-4	Infer Python types for all JSON value types per the mapping in Section 6.	Must
FR-5	Create a separate class for each nested object, with generated names derived from the parent key (e.g. 'address' becomes Address).	Must
FR-6	Allow the user to set the root class name (default 'Root' or inferred, e.g. 'User').	Must
FR-7	Emit correct imports (typing, dataclasses, pydantic) and order classes so dependencies are defined first.	Must
FR-8	Validate generated code by compiling it and instantiating the root class with the original JSON; return status.	Must
FR-9	On validation failure, the agent revises and retries up to 3 times, then returns best result with a warning.	Must
FR-10	Handle arrays: homogeneous (list[T]), empty (list[Any]), arrays of objects (list[ClassName]), mixed (list[A B]).	Must
FR-11	Handle null as Optional[T] / T None; use Any when the type cannot be determined.	Must
FR-12	Convert non-identifier or non-snake_case keys (e.g. 'firstName', 'first-name', '2fa') to valid names; for Pydantic use Field(alias=...); for dataclasses provide a from_dict helper.	Should
FR-13	Accept multiple sample JSON objects and merge them, marking keys missing in some samples as optional.	Should
FR-14	Detect common string formats (ISO date/datetime, UUID, email, URL) and offer richer types.	Should

ID	Requirement	Priority
FR-15	Produce a short decision log explaining naming, nullability, and union choices.	Should
FR-16	Ask a clarifying question when ambiguity affects the result (e.g. field is int in one sample and float in another).	Should
FR-17	Provide copy-to-clipboard and download-as-.py actions.	Should
FR-18	Support top-level JSON arrays by modelling the item type.	Could
FR-19	Optional dataclass flags: frozen, slots, kw_only.	Could

6. Type Mapping Rules

JSON type	Python type	Notes
string	str	May become datetime, date, UUID, EmailStr, HttpUrl when a format is detected (FR-14).
number (integer)	int	If any sample has a decimal for the same key, use float.
number (decimal)	float	
boolean	bool	Checked before int, since bool is a subclass of int in Python.
array	list[T]	T is inferred from items; list[Any] if empty; list[A B] if mixed.
object	Nested class	Class named from parent key, singularized for array items.
null	T None	Optional; default None. Type Any None if only null was seen.

6.1 Example: nested JSON with Pydantic output

Input:

```
{"name": "Avinaash", "address": {"city": "Chennai", "country": "India"}, "tags": ["a", "b"]}
```

Output:

```
from pydantic import BaseModel
```

```
class Address(BaseModel):
    city: str
    country: str
```

```
class User(BaseModel):
    name: str
    address: Address
    tags: list[str]
```

7. Agent Architecture

The agent follows a plan-act-observe loop and keeps a small working state (parsed JSON, inferred schema, chosen format, generated code, error history, decision log).

7.1 Tools available to the agent

Tool	Purpose	Type
parse_json	Parse text; return the object or a precise error location.	Deterministic
infer_schema	Walk the JSON and produce a raw type tree with counts and nullability.	Deterministic
suggest_names	Propose class and field names; sanitize keys; detect keywords/collisions.	LLM + rules

Tool	Purpose	Type
generate_code	Render Dataclass or Pydantic source from the finalized schema plan.	Template-based
validate_code	Compile in a sandbox, import, instantiate root class with the input JSON, compare round-trip.	Sandboxed execution
ask_user	Pose a clarifying question when ambiguity cannot be resolved safely.	Interaction

7.2 Agent responsibilities versus code responsibilities

Type inference and code rendering are handled by reliable deterministic code. The LLM is used where judgement is needed: naming, resolving ambiguity, interpreting validation failures, and writing the explanation. This keeps output correct and cheap while still making the system genuinely agentic.

7.3 Stopping conditions

- Validation passes: return code, decision log, and status 'validated'.
- Three repair attempts fail: return best attempt with status 'unvalidated' and the last error.
- Input invalid or clarification pending: stop and prompt the user.

8. Edge Cases

Case	Expected behaviour
Empty object {}	Generate an empty class (pass) and note it in the log.
Empty array []	list[Any]; ask user for a sample item if they want a precise type.
Key is a Python keyword ('class', 'from')	Rename with trailing underscore; alias in Pydantic.
Key with spaces, dashes, or leading digits	Sanitize to snake_case; keep original via alias / mapping.
Same key name at different depths with different shapes	Distinct class names (e.g. Address, BillingAddress).
Array of mixed objects	Merge into one class with optional fields, or Union if shapes differ substantially.
Very deep nesting or large payloads	Set limits (depth 20, size 1 MB in v1) and show a clear message.
Recursive-looking structures (tree nodes)	Use forward references / string annotations.
Duplicate keys in JSON text	Warn the user; last value wins per Python's parser.
Invalid JSON (single quotes, trailing commas)	Report error with location; do not guess silently.

9. Non-Functional Requirements

Area	Requirement
Correctness	At least 95% of test payloads validate successfully on the first or second attempt.
Performance	Typical payload (under 50 KB) returns in under 10 seconds end to end.
Security	Generated code executes only in a sandbox with no network or file access; input is never executed as code.
Privacy	JSON is processed in memory; not stored or logged beyond the session unless the user opts in. Warn users not to paste secrets.
Code quality	Output is PEP 8 formatted, uses modern typing (list[str], X None), and runs on Python 3.10+.

Area	Requirement
Usability	Result appears alongside input; clear error messages; one-click copy.
Maintainability	Tools are separate modules with unit tests; prompts are versioned.

10. Suggested Tech Stack

Layer	Choice
Language	Python 3.10+
Agent / LLM	Anthropic Claude API with tool use (or any LLM with function calling)
Validation	pydantic v2, dataclasses, ast/compile, restricted subprocess sandbox
Code formatting	black or ruff format on output
UI	Streamlit (web); argparse typer (CLI)
Testing	pytest with a corpus of sample JSON payloads

11. Success Metrics

Metric	Target
Validation pass rate on test corpus (50+ payloads)	95% or higher
Average repair attempts per request	Under 0.5
Correct nested class structure vs hand-written reference	90% or higher match
Median response time	Under 8 seconds
User rating of output usefulness (demo/testing group)	4 out of 5 or higher

12. Milestones

Phase	Scope	Duration
1. Core engine	JSON parsing, type inference, Dataclass/Pydantic generators, nested classes, unit tests.	Week 1
2. Agent loop	LLM tool calling, naming, validate-and-repair loop, decision log.	Week 2
3. Interface	Streamlit UI, format selector, copy/download, error handling.	Week 3
4. Advanced features	Multi-sample merge, format detection, clarifying questions, aliases.	Week 4
5. Evaluation and demo	Test corpus, metrics, documentation, demo video/report.	Week 5

13. Risks and Mitigations

Risk	Mitigation
Project seen as a simple script rather than an agent	Emphasize the tool-use loop, validation, self-repair, and clarification steps; show traces in the demo.
LLM produces invalid code	Render code from templates; LLM only edits the schema plan; always validate before returning.
Unsafe execution of generated code	Sandboxed subprocess, timeouts, no network, no file system writes.

Risk	Mitigation
Ambiguous data leads to wrong types	Multi-sample merge, clarifying questions, and transparent decision log.
Sensitive data pasted into the tool	Clear privacy notice; no persistent storage by default.

14. Future Scope

- Input from JSON Schema or OpenAPI specs; support for YAML and TOML.
- Additional targets: TypedDict, attrs, msgspec, TypeScript interfaces.
- IDE extension or GitHub Action to regenerate models when an API sample changes.
- Learning from user corrections to improve naming preferences.